CERTIK

Security Assessment

Wemix Retainer 126 Governance

CertiK Assessed on Aug 16th, 2024



CertiK Assessed on Aug 16th, 2024

Wemix Retainer 126 Governance

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	EVM Compatible	Formal Verification, Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Solidity	Delivered on 08/16/2024	N/A

CODEBASE

https://github.com/wemixarchive/governance-contract/tree/3abd538e5e4baf3852c5c29dfad3f7dd791812c2/contracts

View All in Codebase Page

Highlighted Centralization Risks

Contract upgradeability

Vulnerability Summary



8	4	0	0	4	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
2	Major	2 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
1	Medium	1 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Minor	2 Resolved, 2 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
1	Informational	1 Resolved	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | WEMIX RETAINER 126 GOVERNANCE

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Findings

CON-01 : Centralized Control of Contract Upgrade

CON-02 : Centralization Related Risks

CON-03 : Incorrect comparison between Vote ID and Ballot ID

GGI-02 : Unused Value

GGI-03 : Potential Out of Array Range

GGI-04 : The check condition does not include `getGovAddress()).ballotLength() == ballotId`

GGI-05 : Logic of `getBallotList()`, `getBallotListAll()` and `getTargetTotalVoters()`

GGI-06 : Test Code Leftover

Optimizations

GGI-01 : Unused Variable

Formal Verification

Considered Functions And Scope

Verification Results

Appendix

Disclaimer

CODEBASE | WEMIX RETAINER 126 GOVERNANCE

Repository

https://github.com/wemixarchive/governance-contract/tree/3abd538e5e4baf3852c5c29dfad3f7dd791812c2/contracts

AUDIT SCOPE | WEMIX RETAINER 126 GOVERNANCE

5 files audited ● 3 files with Acknowledged findings ● 2 files without findings

ID	Repo	File	SHA256 Checksum
GGI	wemixarchive/governance-contract	gateway/GovGatewayImp.sol	2217ead2e5938d5041bc703a027b7d5fb5aeb507b9f425e7a780349cdf193fb2
GIB	wemixarchive/governance-contract	GovImp.sol	11f3d4a0c31b12e2454cda1670e93874c4ed1e0c9d9ab85a69201225120f4a87
BSI	wemixarchive/governance-contract	storage/BallotStorageImp.sol	8f5c09a366ddedefa3590b52500c1fb1218ba0c54ae6a62ff4396955bcecf6e3
GGB	wemixarchive/governance-contract	gateway/GovGateway.sol	5d1ae4ebf8b67c57caa62c8f6fceaa4461a2f1cf470c6cc3f2758b95ae300d57
IGG	wemixarchive/governance-contract	gateway/IGovGateway.sol	0880f4553cefe17352b1e0396d2b8aad62e8ba4a2815258e5812c66c502d9d02

APPROACH & METHODS | WEMIX RETAINER 126 GOVERNANCE

This report has been prepared for Wemix to discover issues and vulnerabilities in the source code of the Wemix Retainer 126 Governance project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | WEMIX RETAINER 126 GOVERNANCE



8	0	2	1	4	1
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Wemix Retainer 126 Governance. Through this audit, we have uncovered 8 issues ranging from different severity levels. Utilizing the techniques of Static Analysis & Manual Review to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
CON-01	**Centralized Control Of Contract Upgrade**	**Centralization**	**Major**	**Acknowledged**
CON-02	**Centralization Related Risks**	**Centralization**	**Major**	**Acknowledged**
CON-03	Incorrect Comparison Between Vote ID And Ballot ID	Logical Issue	Medium	Resolved
GGI-02	Unused Value	Inconsistency	Minor	Resolved
GGI-03	Potential Out Of Array Range	Volatile Code	Minor	Acknowledged
GGI-04	The Check Condition Does Not Include `getGovAddress()).ballotLength() == ballotId`	Volatile Code	Minor	Resolved
GGI-05	Logic Of `getBallotList()` , `getBallotListAll()` And `getTargetTotalVoters()`	Logical Issue	Minor	Acknowledged
GGI-06	Test Code Leftover	Coding Style	Informational	Resolved

CON-01 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	Major	GovImp.sol: 15; gateway/GovGatewayImp.sol: 11; storage/BallotStorageImp.sol: 10	Acknowledged

Description

In the above contracts, the role `owner` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `owner` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (⅔, ⅗) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
  AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

Alleviation

[Wemix Team, 08/16/2024]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

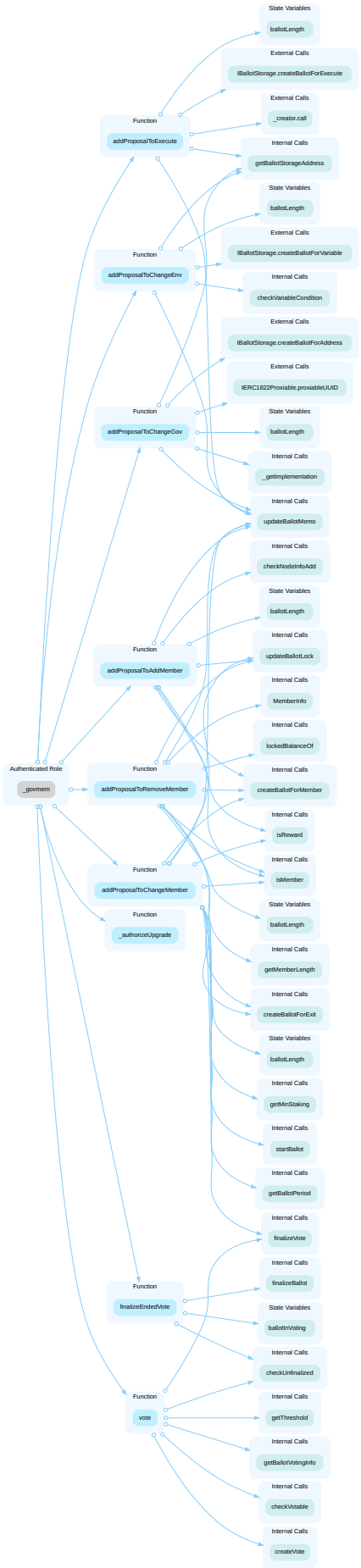
[CertiK, 08/16/2024]: It should be noted that the centralization risk issue still exists. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

CON-02 | CENTRALIZATION RELATED RISKS

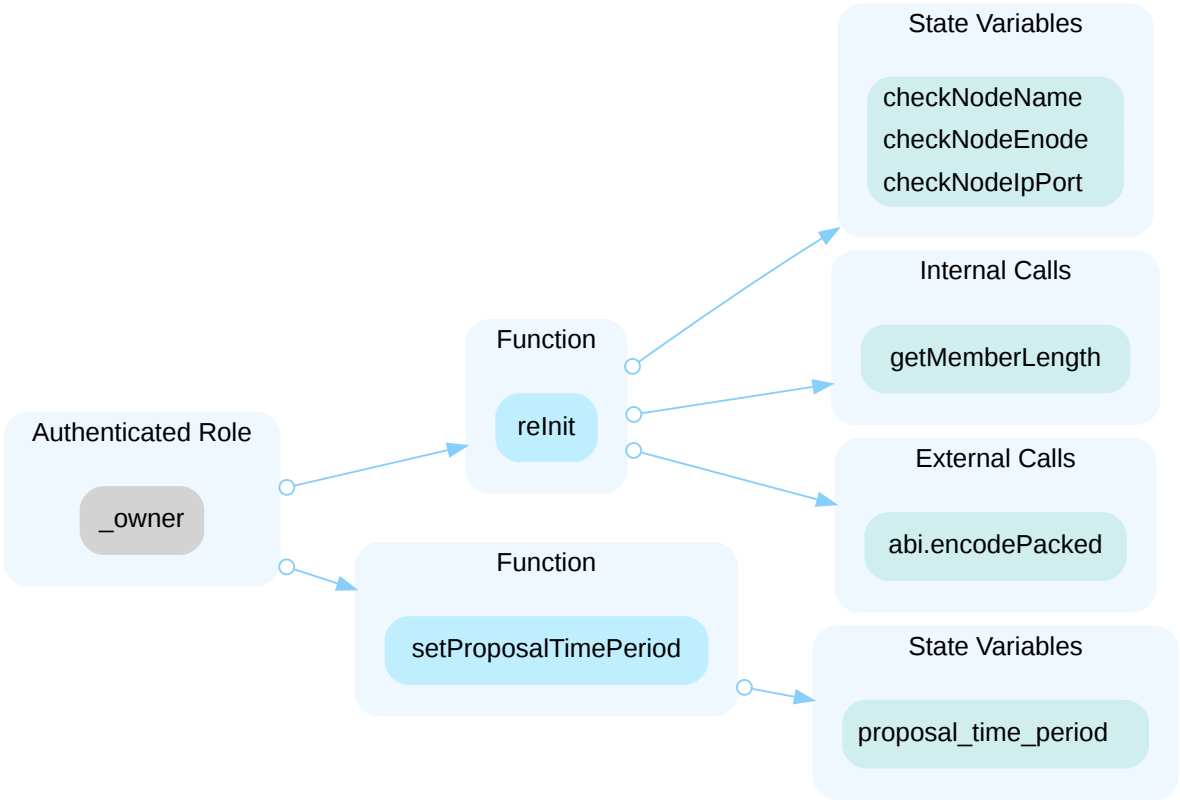
Category	Severity	Location	Status
Centralization	Major	GovImp.sol: 235, 254, 301, 348, 373, 398, 455, 962, 973, 1026, 1104; gateway/GovGatewayImp.sol: 372, 431; storage/BallotStorageImp.sol: 247, 562, 569	Acknowledged

Description

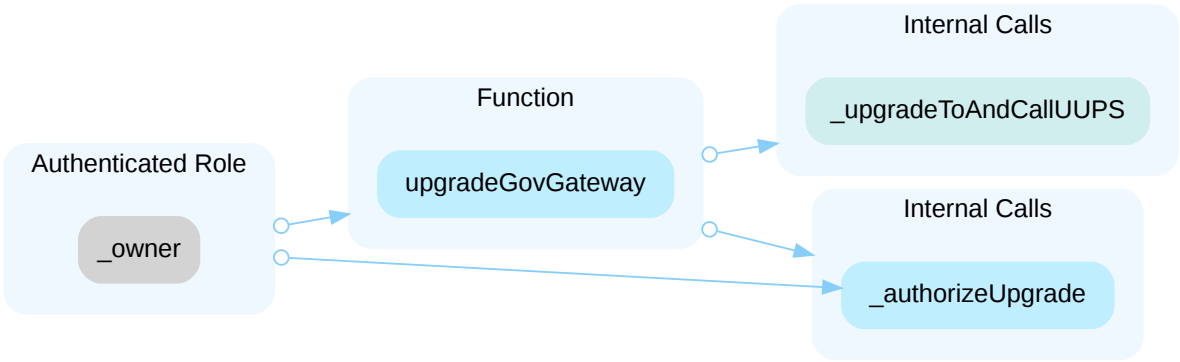
In the contract `GovImp`, the role `_govmem` has authority over the functions shown in the diagram below. Any compromise to the `_govmem` account may allow the hacker to take advantage of this authority and add a proposal to change member, add a proposal for execution, add a proposal to change environment settings, add a proposal to add member, add a proposal to change governance address, authorize contract upgrade, finalize the ended vote, vote on a ballot, and add a proposal to remove member.


Authenticated Role
_govmem
Function
addProposalToExecute
State Variables
ballotLength
External Calls
IBallotStorage.createBallotForExecute
External Calls
_creator.call
Internal Calls
getBallotStorageAddress
Function
addProposalToChangeEnv
State Variables
ballotLength
External Calls
IBallotStorage.createBallotForVariable
Internal Calls
checkVariableCondition
Function
addProposalToChangeGov
External Calls
IBallotStorage.createBallotForAddress
External Calls
IERC1822Proxiable.proxiableUUID
State Variables
ballotLength
Internal Calls
_getImplementation
Internal Calls
updateBallotMemo
Function
addProposalToAddMember
Internal Calls
checkNodeInfoAdd
State Variables
ballotLength
Internal Calls
updateBallotLock
Internal Calls
MemberInfo
Internal Calls
lockedBalanceOf
Function
addProposalToRemoveMember
Internal Calls
createBallotForMember
Internal Calls
isReward
Function
addProposalToChangeMember
Internal Calls
isMember
Function
_authorizeUpgrade
State Variables
ballotLength
Internal Calls
getMemberLength
Internal Calls
createBallotForExt
State Variables
ballotLength
Internal Calls
getMinStaking
Internal Calls
startBallot
Internal Calls
getBallotPeriod
Internal Calls
finalizeVote
Internal Calls
finalizeBallot
Function
finalizeEndedVote
State Variables
ballotInVoting
Internal Calls
checkUnfinalized
Function
vote
Internal Calls
getThreshold
Internal Calls
getBallotVotingInfo
Internal Calls
checkVotable
Internal Calls
createVote

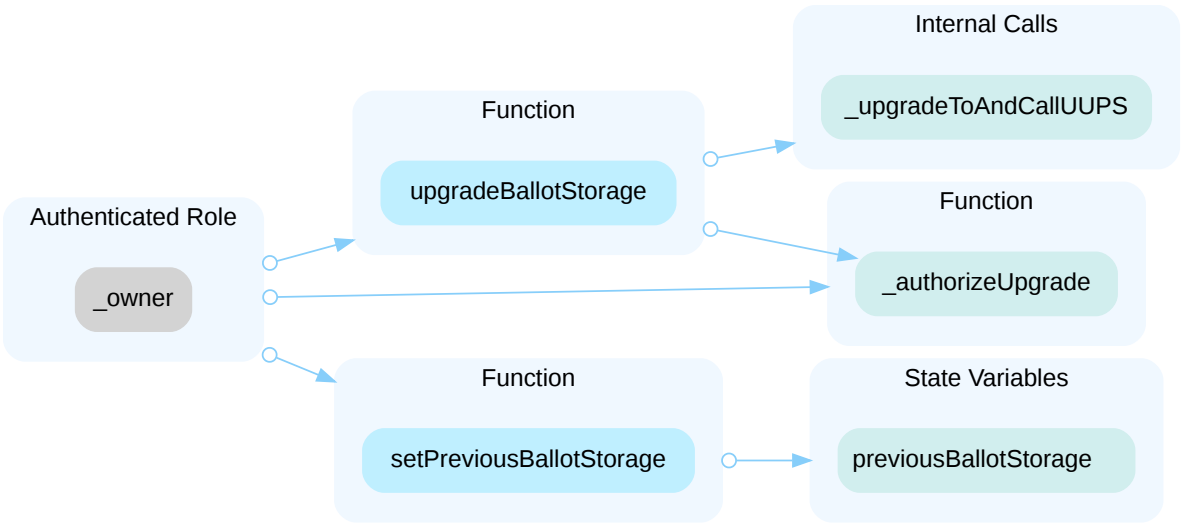
In the contract `GovImp`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and reinitialize node checks, set the proposal time period.



In the contract `GovGatewayImp`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and upgrade the governance gateway implementation or authorize contract upgrades.



In the contract `BallotStorageImp`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and upgrade ballot storage, authorize contract upgrade, and set the previous ballot storage address.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Wemix Team, 08/16/2024]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

[CertiK, 08/16/2024]: It should be noted that the centralization risk issue still exists. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

CON-03 INCORRECT COMPARISON BETWEEN VOTE ID AND BALLOT ID

Category	Severity	Location	Status
Logical Issue	Medium	gateway/GovGatewayImp.sol: 168; storage/BallotStorageImp.sol: 434	Resolved

Description

In the function `getVote()` of the `BallotStorageImp` contract, the input parameter is vote id and the the first return value is the vote id, while the second return value is the ballot id. Therefore, the `id` should be compared with the vote id instead of the ballot id.

```
function getVote(
        uint256 _voteId
    ) public view returns (uint256 voteId, uint256 ballotId, address voter, uint256
decision, uint256 power, uint256 time) {
        require(voteMap[_voteId].voteId == _voteId, "not existed voteId");
        Vote memory _vote = voteMap[_voteId];
        voteId = _vote.voteId;
        ballotId = _vote.ballotId;
        voter = _vote.voter;
        decision = _vote.decision;
        power = _vote.power;
        time = _vote.time;
    }
```

Recommendation

We recommend modifying the code as follows:

```
function isVote(uint256 id, address ballotStorageAddr) public view returns (bool) {
        IGovGateway ballotGateway = IGovGateway(ballotStorageAddr);
        (uint256 voteId, , , , , ) = ballotGateway.getVote(id);

        return id == voteId;
    }
```

Alleviation

The client revised the code and resolved this issue in commit : d6e81d36a16b424e6d0b38acf330f7d5d162d1d8

GGI-02 | UNUSED VALUE

Category	Severity	Location	Status
Inconsistency	Minor	gateway/GovGatewayImp.sol: 208, 249, 267, 338, 342	Resolved

Description

The smart contract contains one or more values that are not used, which can lead to unnecessary complexity and reduced maintainability.

```
208         uint256 ballotLength = govGateway.ballotLength();
```

- The variable `ballotLength` is never used after this assignment.

```
249         uint256 ballotLength = govGateway.ballotLength();
```

- The variable `ballotLength` is never used after this assignment.

```
267         IGovGateway govGateway = IGovGateway(getGovAddress());
```

- The variable `govGateway` is never used after this assignment.

```
338         uint256 ballotLength = govGateway.ballotLength();
```

- The variable `ballotLength` is never used after this assignment.

```
342             IGovGateway ballotGateway = IGovGateway(ballotAddressList[
addressIdx]);
```

- The variable `ballotGateway` is never used after this assignment.

Recommendation

It is advised to ensure that all necessary values are used, and remove redundant values.

Alleviation

The client revised the code and resolved this issue in commit : 57b2f17d6c664f719b134d07885da758600db72d

GGI-03 | POTENTIAL OUT OF ARRAY RANGE

Category	Severity	Location	Status
Volatile Code	Minor	gateway/GovGatewayImp.sol: 199, 243, 307~310, 307~310, 397, 406, 415, 424	Acknowledged

Description

In the function `getVoteListByBallotId()`, within the `for` loop on line 189, if the `ballotId` is not found in the `ballotAddressList`, the `j` will increase by 1. There is a scenario that the `ballotId` is not found in any of the `ballotAddressList`, causing `j` to equal the length of `ballotAddressList`. As a result, line 199 will revert due to an out-of-bounds array access.

The line 243 of the function `getVoteIdListByBallotId()` has the same issue.

In the functions `getBallotList()` and `getBallotListAll()`, if the current ballot id index is not the expected ballot, the `j` will increase by 1, potentially exceeding the length of the ballot address list. Additionally, the new `j` does not invoke the `isBallot()` function to validate the data.

Additionally, the `for` loops in the functions `getBallotMember()`, `getBallotVariable()`, `getBallotAddress()` and `getBallotExit()` exceed the length of the ballot address list.

Recommendation

We recommend adding a check to prevent this scenario from occurring.

Alleviation

[Wemix Team, 08/16/2024]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

GGI-04 THE CHECK CONDITION DOES NOT INCLUDE `getGovAddress()).ballotLength() == ballotId`

Category	Severity	Location	Status
Volatile Code	Minor	gateway/GovGatewayImp.sol: 227	Resolved

Description

In the function `addProposalToAddMember()` of the `GovImp()` contract, the `ballotIdx` is set to the previous `ballotLength` incremented by 1. At the end of this function, the `ballotLength` is updated to `ballotIdx`. Therefore, the maximum ballot id corresponds to the `getGovAddress()).ballotLength()`, and the ballot id starts from 1 instead of 0.

```
235   function addProposalToAddMember(
236           MemberInfo memory info
237       ) external onlyGovMem checkTimePeriod checkLockedAmount checkMemberInfo(
info) returns (uint256 ballotIdx) {
238           require(!isMember(info.staker) && !isReward(info.staker),
"Already member");
239           require(info.staker == info.voter && info.staker == info.reward,
"Staker is not voter");
240           require(checkNodeInfoAdd(info.name, info.enode, info.ip, info.port),
"Duplicated node info");
241           ballotIdx = ballotLength + 1;
242           createBallotForMember(
243               ballotIdx, // ballot id
244               uint256(BallotTypes.MemberAdd), // ballot type
245               msg.sender, // creator
246               ZERO, // old staker address
247               info
248           );
249           updateBallotLock(ballotIdx, info.lockAmount);
250           updateBallotMemo(ballotIdx, info.memo);
251           ballotLength = ballotIdx;
252       }
253
```

Recommendation

We recommend using `require(IGovGateway(getGovAddress()).ballotLength() >= ballotId)` instead of `require(IGovGateway(getGovAddress()).ballotLength() > ballotId)`.

Alleviation

The client revised the code and resolved this issue in commit : 88c38531c7118144266060ff386a1002a4aaac06

GGI-05 LOGIC OF `getBallotList()` , `getBallotListAll()` AND `getTargetTotalVoters()`

Category	Severity	Location	Status
Logical Issue	Minor	gateway/GovGatewayImp.sol: 299	Acknowledged

Description

Using the function `getBallotList()` as an example, the function `getBallotStorageAddressList()` returns the ballot storage address list. The function `getBallotList()` is used to return the ballot list. The function should first loop through the ballot storage address list, then loop through the ballot id list to determine if the ballot Id is valid. If valid, it should be added to the return list. Instead of determining whether the ballot ID is valid, the function simply proceeds to the next ballot storage address in the list. This lack of validation can result in the function continuously incrementing the index by 1, potentially exceeding the length of the ballot storage address list.

```
function getBallotList(uint256 offset, uint256 limit) external view returns
(BallotBasic[] memory ballotList) {
        address[] memory ballotAddressList = getBallotStorageAddressList();
        ballotList = new BallotBasic[](limit - offset + 1);
        uint256 j = 0;

        for (uint256 i = offset; i <= limit; i++) {
            if (isBallot(i, ballotAddressList[j])) {
                ballotList[i - offset] = createBallot(i, ballotAddressList[j]);
            } else {
                j++;
                ballotList[i - offset] = createBallot(i, ballotAddressList[j]);
            }
        }
        return ballotList;
    }
```

Recommendation

We recommend reviewing and modifying the logic to ensure it meets the requirements.

Alleviation

[Wemix Team, 08/16/2024]: Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement.

GGI-06 | TEST CODE LEFTOVER

Category	Severity	Location	Status
Coding Style	Informational	gateway/GovGatewayImp.sol: 4, 91, 101, 107, 114, 160, 191, 194, 198, 235, 238, 242, 280, 321, 340, 350, 357	Resolved

Description

In contract `GovGatewayImp`, the hardhat console is used for debugging and could be omitted in the production code.

Recommendation

We recommend removing the aforementioned code for simplicity and readability.

Alleviation

The client revised the code and resolved this issue in commit : abfb6e1deb219daf9ffa3ee54fa8d71b55b695f0

OPTIMIZATIONS | WEMIX RETAINER 126 GOVERNANCE

ID	Title	Category	Severity	Status
GGI-01	Unused Variable	Gas Optimization	Optimization	Acknowledged

GGI-01 | UNUSED VARIABLE

Category	Severity	Location	Status
Gas Optimization	Optimization	gateway/GovGatewayImp.sol: 73	Acknowledged

Description

The `registry` variable is declared but not used in the contract.

Recommendation

We recommend removing it if there is no plan for further usage.

Alleviation

The client acknowledged this finding, but decided to retain the code base unchanged.

FORMAL VERIFICATION | WEMIX RETAINER 126 GOVERNANCE

Formal guarantees about the behavior of smart contracts can be obtained by reasoning about properties relating to the entire contract (e.g. contract invariants) or to specific functions of the contract. Once such properties are proven to be valid, they guarantee that the contract behaves as specified by the property. As part of this audit, we applied formal verification to prove that important functions in the smart contracts adhere to their expected behaviors.

Considered Functions And Scope

In the following, we provide a description of the properties that have been used in this audit. They are grouped according to the type of contract they apply to.

Verification of Standard Ownable Properties

We verified *partial* properties of the public interfaces of those token contracts that implement the Ownable interface. This involves:

- function `owner` that returns the current owner,
- functions `renounceOwnership` that removes ownership,
- function `transferOwnership` that transfers the ownership to a new owner.

The properties that were considered within the scope of this audit are as follows:

Property Name	Title
ownable-transferownership-correct	Ownership is Transferred
ownable-owner-succeed-normal	`owner` Always Succeeds
ownable-renounceownership-correct	Ownership is Removed
ownable-renounce-ownership-is-permanent	Once Renounced, Ownership Cannot be Regained

Verification Results

In the remainder of this section, we list all contracts where formal verification of at least one property was not successful. There are several reasons why this could happen:

- False: The property is violated by the project.
- Inconclusive: The proof engine cannot prove or disprove the property due to timeouts or exceptions.
- Inapplicable: The property does not apply to the project.

Detailed Results For Contract BallotStorageImp (contracts/storage/BallotStorageImp.sol) In Commit 3abd538e5e4baf3852c5c29dfad3f7dd791812c2

Verification of Standard Ownable Properties

Detailed Results for Function `transferOwnership`

Property Name	Final Result	Remarks
ownable-transferownership-correct	True	

Detailed Results for Function `owner`

Property Name	Final Result	Remarks
ownable-owner-succeed-normal	True	

Detailed Results for Function `renounceOwnership`

Property Name	Final Result	Remarks
ownable-renounceownership-correct	True	
ownable-renounce-ownership-is-permanent	Inconclusive	

APPENDIX | WEMIX RETAINER 126 GOVERNANCE

Finding Categories

Categories	Description
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

Details on Formal Verification

Some Solidity smart contracts from this project have been formally verified. Each such contract was compiled into a mathematical model that reflects all its possible behaviors with respect to the property. The model takes into account the semantics of the Solidity instructions found in the contract. All verification results that we report are based on that model.

The following assumptions and simplifications apply to our model:

- Certain low-level calls and inline assembly are not supported and may lead to a contract not being formally verified.
- We model the semantics of the Solidity source code and not the semantics of the EVM bytecode in a compiled contract.

Formalism for property specifications

All properties are expressed in a behavioral interface specification language that CertiK has developed for Solidity, which allows us to specify the behavior of each function in terms of the contract state and its parameters and return values, as well as contract properties that are maintained by every observable state transition. Observable state transitions occur when the contract's external interface is invoked and the invocation does not revert, and when the contract's Ether balance is changed by the EVM due to another contract's "self-destruct" invocation. The specification language has the usual Boolean connectives, as well as the operator `\old` (used to denote the state of a variable before a state transition), and several types of specification clause:

Apart from the Boolean connectives and the modal operators "always" (written `[]`) and "eventually" (written `<>`), we use the following predicates to reason about the validity of atomic propositions. They are evaluated on the contract's state whenever a discrete time step occurs:

- `requires [cond]` - the condition `cond`, which refers to a function's parameters, return values, and contract state variables, must hold when a function is invoked in order for it to exhibit a specified behavior.
- `ensures [cond]` - the condition `cond`, which refers to a function's parameters, return values, and both `\old` and current contract state variables, is guaranteed to hold when a function returns if the corresponding requires condition held when it was invoked.
- `invariant [cond]` - the condition `cond`, which refers only to contract state variables, is guaranteed to hold at every observable contract state.
- `constraint [cond]` - the condition `cond`, which refers to both `\old` and current contract state variables, is guaranteed to hold at every observable contract state except for the initial state after construction (because there is no previous state); constraints are used to restrict how contract state can change over time.

Description of the Analyzed Ownable Properties

Properties related to function `transferOwnership`

ownable-transferownership-correct

Invocations of `transferOwnership(newOwner)` must transfer the ownership to the `newOwner`.

Specification:

```
ensures this.owner() == newOwner;
```

Properties related to function `owner`

ownable-owner-succeed-normal

Function `owner` must always succeed if it does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `renounceOwnership`

ownable-renounce-ownership-is-permanent

The contract must prohibit regaining of ownership once it has been renounced.

Specification:

```
constraint \old(owner()) == address(0) ==> owner() == address(0);
```

ownable-renounceownership-correct

Invocations of `renounceOwnership()` must set ownership to address(0).

Specification:

```
ensures this.owner() == address(0);
```

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

